02 - Decisions

Every program we have written so far does exactly the same thing every time it runs. This notebook is about programs that **decide**:

- Boolean values and comparison operators
- Membership: `in` and `not in`
- Logical operators, precedence and short-circuiting
- Truthiness
- Analyzing strings
- `if`, `else` and `elif`
- Nested conditionals

Roughly the first half is about writing a *condition*, and the second half is about acting on one.

Three kinds of control structure

Programs are built out of three kinds of control structure:

- **Sequential** — the statements run one after another, from top to bottom. This is the default, and it is all we have used so far.
- **Selection** — the program picks *one* of several blocks of code to run, depending on a condition. That is this notebook.
- **Iterative** — a block of code is repeated. You have not met loops yet.

Selection is what turns a calculation into a program: it lets the same code react differently to different inputs.

Here is where we are heading. Read it, but do not worry about the syntax yet.

In [1]:
```
score = 87

if score >= 80:
    print("Grade: A or B")
elif score >= 60:
    print("Grade: C or D")
else:
    print("Grade: E or F")
```

```
Grade: A or B
```

The *structure* above is not the hard part — it is three keywords, a colon and some indentation, and you will have it memorized by the end of the session.

Loading [MathJax]/extensions/Safe.js

The hard part is the **condition**: the piece between `if` and the colon. A program with the right structure and a wrong condition runs perfectly happily and gives wrong answers, every time, silently. So we start with conditions.

Boolean values

A condition has to come out either true or false, so Python has a data type with exactly two values: `True` and `False`. This is the **boolean** type, written `bool`.

In [2]:
```
is_raining = True
is_sunny = False

print(is_raining)
print(type(is_raining))
```

```
True
<class 'bool'>
```

Two things to notice. `True` and `False` are written with a capital first letter, and they are *not* strings — there are no quotation marks around them. `"True"` is a piece of text that happens to spell a word; `True` is a value.

Comparison operators

We rarely type `True` and `False` by hand. We produce them by comparing values, using the operators you already know from mathematics:

Operator	Meaning	Example
`==`	Equal to	`x == y`
`!=`	Not equal to	`x != y`
`>`	Greater than	`x > y`
`>=`	Greater than or equal to	`x >= y`
`<`	Less than	`x < y`
`<=`	Less than or equal to	`x <= y`

Every one of them returns a boolean.

In [3]:
```
x = 10
y = 3

print(x == y)
print(x != y)
print(x > y)
print(x >= 10)
```

Loading [MathJax]/extensions/Safe.js

```
print(x < y)
print(x <= y)
```

```
False
True
True
True
False
False
```

> ⚠️ **Warning:** `=` and `==` are different operators. A single `=` *assigns* a value to a name; a double `==` *asks* whether two values are equal. Confusing them is a common beginner error in Python.

Comparing strings

Comparison works on text as well as numbers. For two strings to be equal they must contain exactly the same characters, in the same order — and case counts, and so do spaces.

In [4]:
```
print("bergen" == "bergen") #true
print("bergen" == "Bergen") # false
print("bergen" == "bergen ")# false
```

```
True
False
False
```

When the string comes from a person rather than from our own code, that strictness is a nuisance: somebody typing a city name should not have to guess our capitalization. The fix is to clean up both sides before comparing, using the string methods from last time.

In [5]:
```
answer = "  Bergen  "

print(answer == "bergen") #false
print(answer.strip().lower() == "bergen") #true
```

```
False
True
```

The `<` and `>` operators work on strings too, where they usually mean *alphabetical order*. Python orders characters by their internal numeric code, and in that ordering **every** capital letter comes before **every** lowercase letter.

In [6]:
```
print("apple" < "banana") # true
print("Apple" < "apple") # true because capital letters are before lower
print("Zebra" < "apple") # true because capital letters are before lower
```

```
True
True
```

Loading [MathJax]/extensions/Safe.js

> 💡 **Tip:** `"Zebra" < "apple"` is `True`, which is not what anyone means by alphabetical. If you are ordering text that came from outside your program, case-fold it first — compare `a.lower() < b.lower()`.

Comparing floats

Recall that floats have limited precision, so calculations can accumulate tiny errors. That is harmless when we round for display, but it is not harmless when we test for equality.

In [7]:
```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3) #false
```

```
0.30000000000000004
False
```

The sum really is `0.30000000000000004`, and `==` is asking an exact question, so the honest answer is `False`. Every language that uses floats behaves this way.

There are two ways out. Round both sides to a number of decimals you actually care about, or ask whether the two values are *close enough* rather than identical.

In [8]:
```
total = 0.1 + 0.2

print(round(total, 3) == 0.3) # true
print(abs(total - 0.3) < 0.000001) #true
```

```
True
True
```

> ⚠️ **Warning:** Never test two *computed* floats with `==`. Whole numbers are safe — integers are stored exactly — so `10 == 10` and `len(word) == 6` are fine.

Chained comparisons

Checking that a value falls inside a range is so common that Python lets us write it the way we would write it in mathematics.

In [9]:
```
score = 87

print(0 <= score <= 100) #true
print(90 <= score <= 100) #false
```

```
True
False
```

Loading [MathJax]/extensions/Safe.js

is

Last time we saw that a variable is a *name attached to an object*, and that two names can point at the same object. Python has an operator for asking about that directly.

`==` asks "do these have the same value?". `is` asks "are these the same object?".

In [10]:
```
lst_a = [1, 2, 3]
lst_b = [1, 2, 3]
lst_c = lst_a

print(lst_a == lst_b) #true
print(lst_a is lst_b) #false, they are not stored in the same place in memor
print(lst_a is lst_c) #true
```

```
True
False
True
```

`lst_a` and `lst_b` have identical contents but they are two separate lists, so `==` is `True` and `is` is `False`. `lst_c` is just another name for `lst_a`, so `is` is `True`.

> ⚠ **Warning:** Use `==` to compare values. `is` has essentially one everyday use, which is testing against `None`: `if result is None:`. Using `is` to compare numbers or strings sometimes appears to work and then fails without warning.

Your turn

A patient's temperature has been measured:

```
reading = 37.8
```

A reading counts as normal when it is between 36.5 and 37.5 degrees, inclusive at both ends.

Write and display an expression that is `True` when the reading is normal.

Check it on 36.5 and on 38.0.

In [19]:
```
reading = 36.5

print(36.5 <= reading <= 37.5)
```

```
True
```

Membership: `in` and `not in`

Loading [MathJax]/extensions/Safe.js

Two more operators that produce a boolean. They ask whether a value occurs inside a sequence.

In [20]:
```
valid_grades = ["A", "B", "C", "D", "E", "F"]

print("B" in valid_grades) #true
print("G" in valid_grades) #false
print("G" not in valid_grades) #true
```

```
True
False
True
```

This is by far the tidiest way to check a value against a list of allowed options, and we will use it constantly for exactly that.

A string is a sequence of characters, so `in` works there too — where it asks whether one string occurs *inside* another. Like everything else about strings, it is case-sensitive.

In [21]:
```
city = "Bergen"

print("erg" in city) #true
print("berg" in city) #false
print("berg" in city.lower()) #true
```

```
True
False
True
```

Combined with indexing, this gives a compact way to ask what kind of character something is.

In [ ]:
```
code = "NO-2026"

print(code[0] in "0123456789") #false
print(code[-1] in "0123456789") #true. the last character is a number (-1)
```

```
False
True
```

On a dictionary, `in` checks the **keys**, never the values.

In [ ]:
```
temperatures = {"mon": 20, "tue": 18, "wed": 19}

print("mon" in temperatures) #true
print(20 in temperatures) #false
```

```
True
False
```

If you want the values you have to say so.

In [27]:
```
print(20 in temperatures.values()) # cool
```

Loading [MathJax]/extensions/Safe.js

```
True
```

> 💡 **Tip:** This is the safe way to read from a dictionary. Asking for a key that does not exist raises a `KeyError` and stops the program, so check first: look up `record["grade"]` only once you know `"grade" in record`.

Your turn

Copy the dictionary below into your solution cell:

```
populations = {
    "Oslo"     : 717710,
    "Bergen"   : 291940,
    "Trondheim": 214565,
    "Stavanger": 149048
}
```

Write and display one expression that checks whether Tromsø is one of the cities in the dictionary, and one that checks whether any city has exactly 214565 inhabitants.

In [31]:
```
populations = {
    "Oslo"     : 717710,
    "Bergen"   : 291940,
    "Trondheim": 214565,
    "Stavanger": 149048
}

print(f"Tromsø is one of the cities? {"Tromsø" in populations}")
print(f"Any city has 214565 inhabitants? {214565 in populations.values()}")
```

```
Tromsø is one of the cities? False
Any city has 214565 inhabitants? True
```

Logical operators

Real conditions usually combine several questions. There are three operators for that: `and`, `or` and `not`.

- `A and B` is `True` only when **both** sides are true
- `A or B` is `True` when **at least one** side is true
- `not A` flips true to false and false to true

In [33]:
```
print(True and True) #true
print(True and False) #false
print(False and False) #false
```

Loading [MathJax]/extensions/Safe.js

```
True
False
False
```

In [34]:
```
print(True or True) #true
print(True or False) #true
print(False or False) #false
```

```
True
True
False
```

In [35]:
```
print(not True) #false
print(not False) #true
```

```
False
True
```

In practice we never write them on bare `True` and `False` — we write them on comparisons.

In [36]:
```
age = 25
income = 320_000

print(age >= 18 and income >= 300_000) #true
print(age >= 30 or income >= 300_000) #true
print(not age >= 30) #true
```

```
True
True
True
```

Precedence: what Python does first

Notice that the cell above needed no brackets. That is because Python has a fixed order of operations, the same way arithmetic does:

1. arithmetic (`*` , `+` , ...)
2. comparisons (`==` , `<` , `in` , ...)
3. `not`
4. `and`
5. `or`

So `not age >= 30` is read as `not (age >= 30)` , which is what we wanted. But the same rule means that two expressions built from identical pieces can mean different things.

In [37]:
```
a = True
b = False

print(not a and b) #false
print(not (a and b)) #true
```

Loading [MathJax]/extensions/Safe.js

```
False
True
```

`not` binds tighter than `and`, so the first line is `(not a) and b`. The brackets in the second line change which operation happens last, and the answers differ.

The one that catches people out most often is that **`and` binds tighter than `or`**.

In [38]:
```python
is_member = False
age = 15
has_ticket = True

# Reads as: (is_member and age >= 18) or has_ticket
print(is_member and age >= 18 or has_ticket) #true

# A different rule entirely
print(is_member and (age >= 18 or has_ticket)) #false
```

```
True
False
```

> 💡 **Tip:** House rule for this course: as soon as a condition mixes `and` with `or`, or puts `not` in front of something compound, write the brackets. They cost two characters and remove all doubt.

One last piece of logic worth knowing, because it turns awkward conditions into readable ones. Pushing a `not` inside brackets flips both the comparisons and the connective:

In [39]:
```python
score = 105

print(score < 0 or score > 100) # true
print(not (0 <= score and score <= 100)) #true
```

```
True
True
```

Both say *"the score is out of range"*. But the first one says it in the way a person would.

Your turn

A cinema gives a discount to **students under 30**, and to **everyone over 66**.

```
is_student = False
age = 70

A:  is_student and age < 30 or age > 66
B:  is_student and (age < 30 or age > 66)
C:  (is_student and age < 30) or age > 66
```

Loading [MathJax]/extensions/Safe.js

> Run all three expressions on those values. Two of them agree and one does not. Which one implements the rule as it is stated above?

In [43]:
```
is_student = True
age = 30

discount = (is_student and age < 30) or age > 66

print("Discount? ", discount)
```

```
Discount?  False
```

Truthiness

Everything we have written so far produced a real `True` or `False`. But Python is more relaxed than that: it will accept *any* value where a condition is expected, and it has a rule for deciding what counts as true. The whole rule is: **zero is false, empty is false, everything else is true.**

The `bool` function shows us that rule directly.

In [44]:
```
print(bool(0)) # false
print(bool(0.0)) # false
print(bool("")) # false
print(bool([])) # false
print(bool({})) # false
print(bool(None)) # false
```

```
False
False
False
False
False
False
```

In [45]:
```
print(bool(1)) # true
print(bool(-3)) # true
print(bool("Bergen")) # true
print(bool("0")) # true
print(bool(" ")) #true
print(bool([0])) # true
```

```
True
True
True
True
True
True
```

Values that are false are called *falsy*, and values that are true are called *truthy*. Two of the results above are worth memorizing, because they are where the rule bites:

Loading [MathJax]/extensions/Safe.js

- `"0"` is a string with one character in it. It is not empty, so it is truthy.
- `" "` is a string with one character in it as well. Also truthy.

Truthiness explains a bug that every beginner writes at least once. Here is a condition that looks like it says *"the grade is A or B"*.

In [46]:
```
grade = "F"

print(grade == "A" or "B") # "B"
```

```
B
```

It printed `B` — not `True`, not `False`, and certainly not a correct answer for a grade of F.

Two things are happening. First, precedence: comparison binds tighter than `or`, so Python reads it as `(grade == "A") or ("B")`. Second, `and` and `or` do not return `True` / `False` at all — they return **one of the two values they were given**.

In [47]:
```
print(False or "B") # "B"
print(bool(False or "B")) # True
```

```
B
True
```

So the condition becomes `False or "B"`, which is `"B"`, which is truthy. The condition is true for every grade in existence.

Each side of an `and` or an `or` has to be a complete question. Both of these are correct:

In [48]:
```
grade = "F"

print(grade == "A" or grade == "B") # False
print(grade in ("A", "B")) # False
```

```
False
False
```

Your turn

A survey form records what each person filled in. A text field left blank arrives as an empty string, and a question that was never answered at all is stored as `None`, Python's way of saying "no value":

```
name  = ""
score = None
```

Write and display one expression that is `True` when the person did not type a name, and one that is `True` when the person did not enter a score.

Loading [MathJax]/extensions/Safe.js

> Check both on a person who typed `Ada` and scored `0`.

In [77]:
```
name  = "Ada"
score = 0

empty_name = name.strip() is None
zero_score = score is None

print(empty_name)
print(zero_score)
```

```
False
False
```

Short-circuiting

`and` and `or` stop evaluating as soon as the answer is certain:

- in `A and B`, if `A` is false the answer is already false, so `B` **is never evaluated**
- in `A or B`, if `A` is true the answer is already true, so `B` **is never evaluated**

That is not a curiosity. It is what lets us write a test that would otherwise crash.

In [55]:
```
code = ""

# len(code) > 0 is False, so code[0] is never evaluated
print(len(code) > 0 and code[0] == "-") # False
```

```
False
```

In [ ]:
```
# IndexError
code = ""

print(code[0] == "-" and len(code) > 0) # Error
```

```
---------------------------------------------------------------------------

IndexError                                Traceback (most recent call last)

Cell In[56], line 4

      1 # IndexError

      2 code = ""

----> 4 print(code[0] == "-" and len(code) > 0) # False



IndexError: string index out of range
```

The two cells contain the same two tests. Swapping them turns a working program into a

Loading [MathJax]/extensions/Safe.js

se in the second one Python has to look at `code[0]` before it ever

reaches the length check — and there is no character 0 in an empty string.

> ⚠️ **Warning:** The order of the parts of a condition matters. Put the test that makes the rest *safe* first.

Your turn

Copy the dictionary below into your solution cell:

```
record = {"name": "Ada", "score": 88}
```

Write and display a single expression that is `True` when the record's grade is `A`, and `False` rather than a crash when the record has no grade at all.

Check it on a record that does have a grade:

```
record["grade"] = "B"
```

In [94]:
```
record = {"name": "Ada", "score": 88, "grade": "A"}

print("grade" in record and record["grade"] == "A")
```

```
True
```

Analyzing strings

A lot of what a program has to decide is *"is this piece of text the right shape?"*. Strings come with a set of methods that ask exactly that sort of question and hand back a boolean, so they drop straight into a condition.

Method	`True` when
`.startswith(s)`	the string starts with `s`
`.endswith(s)`	the string ends with `s`
`.isdigit()`	every character is a digit, and there is at least one
`.isalpha()`	every character is a letter, and there is at least one
`.isspace()`	every character is whitespace, and there is at least one
`.islower()`	every cased character is lowercase, and there is at least one
`.isupper()`	every cased character is uppercase, and there is at least one

In [62]:
```
reference = "NO-2026-0147"

print(reference.startswith("NO"))
```

Loading [MathJax]/extensions/Safe.js

```
print(reference.endswith("0147"))
print(reference.startswith("no"))
```

```
True
True
False
```

In [63]:
```
print("2026".isdigit())
print("2026".isalpha())
print("Bergen".isalpha())
print("   ".isspace())
print("bergen".islower())
```

```
True
False
True
True
True
```

`.isdigit()` does less than it looks like it does

`.isdigit()` is tempting as a way of checking that somebody typed a number. But it has some important limitations.

In [64]:
```
print("42".isdigit())
print("-42".isdigit())
print("3.14".isdigit())
print(" 42".isdigit())
print("".isdigit())
```

```
True
False
False
False
False
```

A minus sign is not a digit. A decimal point is not a digit. A space is not a digit. And an empty string has nothing to check, so the answer is `False`.

`.isdigit()` answers exactly one question: *is this one or more digits and nothing else?* That is the right question for a postal code, a house number or a count of items. It is the wrong question for a temperature, a price, or anything that can be negative or have decimals.

A trap worth seeing once

Suppose we want to know whether a password contains at least one uppercase letter. `.islower()` looks like the answer: if the string is *not* all lowercase, surely there must be a capital in it?

In [65]:
```
print("copycat1337".islower())
print(not "CopyCat1337".islower())
```

Loading [MathJax]/extensions/Safe.js

```
print(not "1337".islower())
```

```
False
True
True
```

The third line is wrong. `"1337"` contains no uppercase letter — it contains no letters at all — and yet the test says it does.

Look again at the description in the table: *every cased character is lowercase, and there is at least one***. `"1337"` has no cased characters, so `.islower()` is `False`, and `not` turns that into a confident wrong answer.

A test that does work: a string contains an uppercase letter exactly when lowercasing it **changes** it.

In [ ]:
```
print("copycat1337" != "copycat1337".lower()) # False
print("CopyCat1337" != "CopyCat1337".lower()) # True
print("1337" != "1337".lower()) # False
```

```
False
True
False
```

> 💡 **Tip:** Whenever a method's description contains the words "and there is at least one", test it on the empty string and on a string containing none of the thing. This will quickly reveal potential issues with the method.

Your turn

Copy the variable below into your solution cell:

```
reference = "no-2026-0147"
```

A booking reference is valid when it starts with `NO` — in either upper or lower case — **and** its last four characters are digits.

Store each of the two rules in its own variable, combine them into a variable called `is_valid`, and display all three variables.

In [79]:
```
reference = "no-2026-0147"

no = reference.lower().startswith("no")
last_four_digits = reference[-4:].isdigit()

is_valid = no and last_four_digits

print(is_valid)
```

Loading [MathJax]/extensions/Safe.js

`if` statements

We can now write conditions. Time to use one.

```
if <condition>:
    <statements>
```

The header starts with `if`, holds the condition, and ends with a **colon**. The block underneath is **indented**, and it runs only when the condition is true. Positron indents for you when you press Enter after the colon.

In [80]:
```
score = 100

if score == 100:
    print("Full score!")
```

```
Full score!
```

Change `score` to 70 and run it again: nothing is displayed, and no error occurs. The block was skipped, which is the whole point.

Code that is *not* indented is not part of the block, so it runs either way.

In [81]:
```
score = 70

if score == 100:
    print("Full score!")

print("This line always runs.")
```

```
This line always runs.
```

The indentation is not decoration. In many other languages the block is marked with brackets and the indentation is only a courtesy to the reader; in Python the indentation *is* the marker, so getting it wrong is an error rather than a matter of taste.

In [82]:
```
# IndentationError
score = 100

if score == 100:
    print("Full score!")
```

```
Full score!
```

And this is where confusing `=` with `==` shows up. Python's error message even guesses what you meant.

In [ ]:
```
# SyntaxError
score = 100
```

Loading [MathJax]/extensions/Safe.js

```
if score = 100:
    print("Full score!")
```

```
Full score!
```

`else`

`else` attaches an alternative block, which runs when the condition is false. It takes no condition of its own — it is simply "otherwise".

In [85]:
```
score = 70

if score == 100:
    print("Full score!")
else:
    print("Not full score.")
```

```
Not full score.
```

Exactly one of the two blocks always runs.

Any condition we have built works in the header — comparisons, membership, logical operators, string methods, all of it.

In [86]:
```
grade = "C"

if grade in ["A", "B", "C", "D", "E"]:
    print("You passed.")
else:
    print("You did not pass.")
```

```
You passed.
```

And any statement at all can go in the body, not only `print`.

In [ ]:
```
num = 12

# Halve the number if even; triple and add 1 if odd
if num % 2 == 0:
    num = num // 2
else:
    num = 3 * num + 1

print(num)
```

Storing the answer instead of printing it

A very common pattern is to record a decision in a variable so that a later part of the program can use it. Here it is, written the way most people write it first.

In [ ]:
```
score = 52
```

Loading [MathJax]/extensions/Safe.js

```
if score >= 60:
    passed = True
else:
    passed = False

print(passed)
```

Look closely at what that code does. The condition `score >= 60` is *already* a `True` / `False` value — the four lines around it do nothing except copy it into a variable.

In [ ]:
```
score = 52

passed = score >= 60

print(passed)
```

A variable holding a boolean like this is often called a **flag**. Once you have one, use it directly.

In [ ]:
```
passed = False

if passed:
    print("Approved")
else:
    print("Not approved")
```

> 💡 **Tip:** Write `if passed:`, not `if passed == True:`. The second one compares a boolean to a boolean in order to produce a boolean, and is superfluous.

This is also where truthiness earns its keep. `if name:` is the idiomatic way to say *"if the user actually typed something"*.

In [ ]:
```
name = ""

if name:
    print(f"Hello, {name}!")
else:
    print("You did not enter a name.")
```

> 💡 **Tip:** When both branches do nothing but assign to the same variable, Python offers a one-line form: `status = "pass" if score >= 60 else "fail"`. You will meet it in other people's code. For anything longer than that, write the full `if` / `else`.

Your turn

Loading [MathJax]/extensions/Safe.js

Copy the variable below into your solution cell:

```
temperature = 4.5
```

Then write a short program that:

- stores in a variable called `is_cold` whether the temperature is below 10 degrees — using a single expression, with no `if` statement
- displays `Wrap up warm` if it is cold and `No jacket needed` otherwise, using `is_cold` directly as the condition

Check that it still behaves correctly for a temperature of 10 and of 25.

In [ ]:
```
temperature = 10

is_cold = temperature < 10 or False

if is_cold:
    print("Wrap up warm")
else:
    print("No jacket needed")
```

```
Wrap up warm
```

`elif`: more than two outcomes

`if` / `else` gives us two possible outcomes. When there are more, we chain the tests together with `elif`, which is short for "else if".

In [ ]:
```
score = 85

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
elif score >= 50:
    print("Grade: E")
else:
    print("Grade: F")
```

Read it as a ladder. Python tests the conditions from the top and **stops at the first one that is true** — the remaining conditions are not even evaluated. The `else` at the bottom is the catch-all, and it is worth always having one: it is what saves the program from silently doing nothing when the input is something you did not anticipate.

Loading [MathJax]/extensions/Safe.js

The order of the branches is part of the logic

Here is the same ladder with the branches in the opposite order.

In [ ]:
```
score = 95

if score >= 50:
    print("Grade: E")
elif score >= 70:
    print("Grade: C")
elif score >= 90:
    print("Grade: A")
else:
    print("Grade: F")
```

A student who scored 95 is awarded an E. Nothing failed, nothing warned, and the code looks entirely reasonable — but `score >= 50` is true for 95, so the ladder stopped on the first rung.

When the conditions overlap like this, order them from the **most demanding to the least**. The alternative is to make every branch state its full range, so that only one of them can ever be true and the order stops mattering.

In [ ]:
```
score = 95

if 90 <= score <= 100:
    print("Grade: A")
elif 70 <= score < 90:
    print("Grade: C or B")
elif 50 <= score < 70:
    print("Grade: D or E")
else:
    print("Grade: F")
```

`elif` versus a stack of separate `if`s

These look almost the same, and they are not.

In [ ]:
```
score = 85

if score >= 90:
    print("Grade: A")
if score >= 80:
    print("Grade: B")
if score >= 70:
    print("Grade: C")
else:
    print("Not A, B or C.")
```

Loading [MathJax]/extensions/Safe.js

It prints **both** B and C, because three separate `if` statements are three separate decisions — each one is evaluated, whatever the others did.

And the `else` belongs to the **third** `if` alone. An `else` always attaches to the nearest `if` above it at the same indentation, never to the group.

> 💡 **Tip:** Use `elif` when the branches are alternatives and exactly one should happen. Use separate `if` statements when the tests are genuinely independent and any number of them may apply.

Your turn

An online shop charges for shipping according to the value of the order:

Order value (NOK)	Shipping
1500 or more	free
750 to 1499	49
250 to 749	79
under 250	99

Store an order value (e.g., 800) in a variable, then write an `if` / `elif` chain that displays the shipping cost for it.

Check the boundaries — 1500, 749 and 250 should each land in the band the table says.

In [93]:
```python
def calc_shipping(order_value) -> int:
    if order_value >= 1500:
        return 0
    elif order_value >= 750:
        return 49
    elif order_value >= 250:
        return 79
    else:
        return 99

print("Shipping cost 1500:", calc_shipping(1500))
print("Shipping cost 749:", calc_shipping(749))
print("Shipping cost 250:", calc_shipping(250))
```

```
Shipping cost 1500: 0
Shipping cost 749: 79
Shipping cost 250: 79
```

Loading [MathJax]/extensions/Safe.js

conditionals

An `if` statement can go inside another `if` statement. That is called **nesting**, and it is written by simply indenting one inside the other.

In [ ]:
```
score = 85

if score >= 90:
    print("Grade: A")
else:
    if score >= 80:
        print("Grade: B")
    else:
        print("Grade: C or below")
```

Although this works, it should rarely be written this way. Every extra grade would add another level of indentation, and a six-grade version would run off the side of the screen — which is precisely what `elif` exists to prevent.

> 💡 **Tip:** When every branch is asking about the same variable, flatten it into an `elif` chain.

Your turn

Free shipping requires that the customer is a member **and** that the order is worth at least 500 kroner.

```
is_member = True
order_value = 320
```

Write a *nested* `if` statement — not a single condition joined with `and` — that displays one of three messages: that the customer is not a member, that they are a member but the order is too small, or that the order qualifies for free shipping.

In [95]:
```
is_member = True
order_value = 320

if is_member:
    if order_value >= 500:
        print("Order qualifies for free shipping")
    else:
        print("Customer is a member, but too small order")
else:
    print("Not a member")
```

```
Customer is a member, but too small order
```

When nesting is the right answer

Loading [MathJax]/extensions/Safe.js

Nesting earns its place when the inner question only *makes sense* once the outer one has been answered.

In [ ]:
```
entry = "42"

if entry.isdigit():
    age = int(entry)

    if age >= 18:
        print("Access granted.")
    else:
        print("Too young.")
else:
    print("That is not a whole number.")
```

These two questions are about different things, and they are ordered. We cannot ask whether the age is at least 18 until we know the text is a number at all — `int("hello")` would stop the program. The nesting says exactly that, and it is the same idea as the short-circuit guard from earlier, written with statements instead of operators.

Run the cell again with `entry = "seven"` and with `entry = "16"`, and notice that the three messages are three genuinely different pieces of information.

Or one condition with `and`?

The same check can be written flat.

In [ ]:
```
entry = "seven"

if entry.isdigit() and int(entry) >= 18:
    print("Access granted.")
else:
    print("No access.")
```

Shorter, and perfectly safe — short-circuiting guarantees `int(entry)` is never reached when `.isdigit()` is false. But it has a cost, and the cost is in the `else`: two quite different problems now produce one message. The person who typed `seven` and the person who is 16 are told the same unhelpful thing.

> 💡 **Tip:** Combine with `and` when all you need to know is *whether* the check passed. Nest when you need to tell the user *why* it failed.

Additional resources

Loading [MathJax]/extensions/Safe.js

hon Tutorial — section 4 covers `if` and `elif`

- Truth value testing — the official statement of which values are falsy
- String methods — the full list, including every `.is...()` method and its exact wording
- Automate the Boring Stuff with Python — chapter 2, free to read online
- SKL401 — lessons 2.1, 2.2 and 2.9 cover comparison operators, conditionals and string operations more briefly

Next week: loops — getting a program to repeat a block of code, so that we stop writing the same lines out by hand.

Loading [MathJax]/extensions/Safe.js